Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

[acá va tu titulo]

Tesis de Licenciatura en Ciencias de la Computación

Rafael Romani

Director: [acá va tu director]

Codirector: [acá va tu codirector]

Buenos Aires, 2026

[ACÁ VA TU TITULO]

[Acá va el resumen]

Palabras claves: [acá van las palabras clave]

[acá van los agradecimientos]

Índice general

In this work we are going to analyze transformers as language recognizers. A transformer is a machine that works over an alphabet Σ. It has only one tape from where it reads the input and in an autoregressive way writes in it more tokens. There are two distinguished tokens called stopping symbols. Once the transformer writes one of those it halts. We say that a transformer accepts a word if when it halts the stopping symbol printed is the accepting one (q_{accept}). Correspondingly, a transformer rejects a word if the stopping symbol printed is the rejecting one (q_{reject}).

In each step the transformer maps the content of the tape to a probability distribution over the alphabet. Depending on this distribution the transformer prints the next token. The transformer iterates this process until it halts. These iterations are called chain of thought.

The process of generating the next token consists of two separated steps. First a probability distribution over the alphabet for the token is computed by a process called distribution generation. After that, it is necessary to decide the specific token by a process called token selection. These two algorithms together is what defines the transformer. During this work we will analyze and compare two different options for token selection.

For a more realistic analysis of the transformers, we don't allow an infinite tape. The transformers studied in this work, for each input size n, will have a tape of size $\text{budget}(n)+n$. It's important to remark that, since in every step of the transformer a symbol is written, the maximum number of steps a transformer can do for an input of length n is $\text{budget}(n)$.

The process of distribution generation is dependent on many parameters. These parameters, in the real models, are the ones calculated during the training. We refer to them as θ. During this work we assume that the parameters force the transformer to halt exactly when the budget runs out. We consider the parameters as part of the definition of a transformer.

Write an appen
about this decis
We can always
an extra step o
artifact forcing

We note $\text{TF}(\alpha)$ to the token outputted by one step of the transformer TF when the tape content is $\alpha \in \Sigma^*$. The autoregressive token generation is defined recursively. The token outputted after i steps is $\text{TF}^i(\alpha) := \text{TF}^{i-1}(\alpha\text{TF}(\alpha))$ with base case $\text{TF}^1(\alpha) := \text{TF}(\alpha)$. In other words, the output of the ith step is $\sigma_i = \text{TF}^i(\alpha) = \text{TF}(\alpha\sigma_1 \dots \sigma_{i-1})$ where $\sigma_j \in \Sigma$ for all j.

We note $\text{TF}^*(\alpha)$ for the token written by the transformer when it, i.e. at the end of the computation, when the input is α. As was said before, the transformer only halts when it prints a stopping symbol (q_{accept} or q_{reject}). Therefore, $\text{TF}^*(\alpha) \in \{q_{accept}, q_{reject}\}$.

As stated before, the **distribution generation** is a mapping of the content of the tape to a probability distribution over Σ. There are different algorithms for this process in the literature. All the transformers considered in this work use an algorithm consisting of the following four parts: a token embedding layer (TE), a position encoding layer (PE), an output linear layer (OUTPUT), and a stack of L identical layers, where L is also called the depth of the model. Each decoder layer has two sub-layers: a multi-head self-attention layer (ATTN) and a position-wise fully-connected feed-forward network (FF). Each part and layer has its own trainable parameters. Then we can define the distribution generation algorithm by its parameters $\theta = \{\theta_{\text{TE}}, \theta_{\text{PE}}, \{\theta^l_{\text{ATTN}}, \theta^l_{\text{FF}}\}_{l=0}^{L-1}, \theta_{\text{OUTPUT}}\}$.

Token Embedding: It's a mapping from Σ to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\text{TE}} \in \mathbb{R}^{d\times|\Sigma|}$. For all $a \in \Sigma$, we note $\theta_{\text{TE}}(a) \in \mathbb{R}^d$ to the mapping of token a.

Position Encoding: It's a mapping from the positions of the tape to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\text{PE}} \in \mathbb{R}^{d\times\text{budget}(n)+n}$. For all position of the tape $1 \leq i \leq \text{budget}(n) + n$ we note $\theta_{\text{PE}}(i) \in \mathbb{R}^d$ to the mapping of position i.

Self Attention Layer: It's a mapping from $(\mathbb{R}^d)^a$ to $(\mathbb{R}^d)^a$. Given parameters $\theta_{\text{ATTN}} = (W_Q, W_K, W_V, W_O) \in \mathbb{R}^d \times \mathbb{R}^d \times \mathbb{R}^d \times \mathbb{R}^d$, the layer it's defined by the algorithm 1. This layer is defined only as a single head attention layer because we can simulate 1 multi-head attention layer with any constantly many heads with multiple single-head attention layers.

ing Toyota,
be search for
riginal source.

Algorithm 1 Causal Self-Attention, ATTN

Require: Parameter $\theta_{\text{ATTN}} = (W_Q, W_K, W_V, W_O)$, Input embedding $h = (h_1, \ldots, h_a) \in \mathbb{R}^d \times \cdots \times \mathbb{R}^d$.

Ensure: Output embedding $h' = (h'_1, \ldots, h'_a) := \text{ATTN}_{\theta_{\text{ATTN}}}(h_1, \ldots, h_a)$.

1: $q_i := W_Q h_i,\ k_i := W_K h_i,\ v_i := W_V h_i,\ \forall i \in [a]$
2: $s_i := \text{softmax}(\langle q_i, k_1 \rangle, \ldots, \langle q_i, k_i \rangle) \| (0, \ldots, 0)$.
3: $h'_i := W_O \sum_{j=1}^{a} (s_i)_j v_j$.

Feed Forward Network: It's a mapping from $\mathbb{R}^d$ to $\mathbb{R}^d$. Given $\theta_{\text{FF}} = (W_1, b_1, W_2, b_2) \in \mathbb{R}^{d \times d} \times \mathbb{R}^d \times \mathbb{R}^{d \times d} \times \mathbb{R}^d$, it's defined as $\text{FF}_{theta_{\text{FF}}}(h) := W_2\ \text{relu}(W_1 h + b1) + b2$, where $\text{relu} : \mathbb{R}^d \to \mathbb{R}^d$ replaces each position x_i by $\max(0, x_i)$.

Output Layer: It's a mapping from $\mathbb{R}^d$ to a probability distribution over Σ. Given $\theta_{\text{OUTPUT}} \in \mathbb{R}^{|\Sigma| \times d}$, it's defined as $\text{OUTPUT}_{\theta_{\text{OUTPUT}}}(h) := \text{softmax}(\theta_{\text{OUTPUT}} h)$.

All these parts come together for the distribution generator process in the algorithm 2.

Algorithm 2 Distribution generator

Require: Transformer parameter $\theta = (\theta_{\text{TE}}, \theta_{\text{PE}}, \{\theta^l_{\text{ATTN}}, \theta^l_{\text{FF}}\}_{l=0}^{L-1}, \theta_{\text{OUTPUT}})$ and tokens of the tape $\alpha \in \Sigma^{|\alpha|}$.

Ensure: Output distribution $p_\theta(\cdot|\alpha)$.

1: $h_i^{(0)} \leftarrow \theta_{\text{TE}}(\alpha_i) + \theta_{\text{PE}}(i),\ \forall i \in |\alpha|$
2: **for** $l = 0, \ldots, L-1$ **do**
3: $\quad (h_1^{(l+0,5)}, \ldots, h_{|\alpha|}^{(l+0,5)}) \leftarrow (h_1^{(l)}, \ldots, h_{|\alpha|}^{(l)}) + \text{ATTN}_{\theta^{(l)}_{\text{ATTN}}}(h_1^{(l)}, \ldots, h_{|\alpha|}^{(l)})$
4: $\quad h_i^{(l+1)} \leftarrow h_i^{(l+0,5)} + \text{FF}_{\theta^{(l)}_{\text{FF}}}(h_i^{(l+0,5)}),\ \forall i \in |\alpha|$
5: **end for**
6: $p_\theta(\cdot|\alpha) \leftarrow \text{OUTPUT}_{\theta_{\text{OUTPUT}}}(h_{|\alpha|}^{(L)})$

The second component of a transformer is the **token selector**. This algorithm consumes the probability distribution computed by the distribution generator and returns a token.

Lets $p : \Sigma \to [0, 1]$ be the output of the distribution generator. In this work we will study two selectors. The first one always returns the token with the largest probability (argmax over p, where p is the output of the distribution generator). The other, works non-deterministically. It returns the token $\sigma \in \Sigma$ with probability $p(\sigma)$.

We call a transformer deterministic when the token selector is the one taking argmax over the distribution. We say that a family of deterministic transformers $\{\text{TF}_n\}_{n \in \mathbb{N}}$ recognize a language $\mathcal{L}$ when:

$$\alpha \in \mathcal{L} \iff \text{TF}^*_{|\alpha|}(\alpha) = q_{accept}$$

Given two functions $T(n)$ and $d(n)$ we define the complexity class $\text{CoT}[T(n), d(n)]$. Informally, a language $\mathcal{L}$ is in $\text{CoT}[T(n), d(n)]$ iff there is a deterministic family of trans-

formers with constant depth, embedding size $d(n)$ and budget $T(n)$ that recognizes it. Formally, we say a language $\mathcal{L}$ is in $\text{CoT}[T(n), d(n)]$ iff there is an integer L and two functions $T'(n) = O(T(n)), d'(n) = O(d(n))$, such that for every positive integer n, there is an L-layer deterministic transformer, denoted by TF_n with embedding size $d'(n)$, that can output $\mathcal{L}(\alpha)$ given any input α in Σ^n, using $T'(n)$ steps of chain of thought.

When TF is a deterministic transformer, $\text{TF}^i(\alpha)$ is an exact symbol for all $\alpha \in \Sigma^*$ and $i \in \mathbb{N}$. In counterpart, when the token selection process of the transformer samples the distribution, it becomes a random variable. We call probabilistic transformers to the transformers that sample the distribution for selecting the next token. We say that a family of probabilistic transformers $\{\text{TF}_n\}_{n\in\mathbb{N}}$ recognize a language $\mathcal{L}$ when:

$$\alpha \in \mathcal{L} \iff \Pr[\text{TF}^*_{|\alpha|}(\alpha) = q_{accept}] > 2/3$$

Given two functions $T(n)$ and $d(n)$ we define the complexity class $\text{PCoT}[T(n), d(n)]$. Informally, a language $\mathcal{L}$ is in $\text{PCoT}[T(n), d(n)]$ iff there is a probabilistic family of transformers with constant depth, embedding size $d(n)$ and budget $T(n)$ that recognizes it. Formally, we say a language $\mathcal{L}$ is in $\text{PCoT}[T(n), d(n)]$ iff there is an integer L and two functions $T'(n) = O(T(n)), d'(n) = O(d(n))$, such that for every positive integer n, there is an L-layer probabilistic transformer, denoted by TF_n with embedding size $d'(n)$, that can output $\mathcal{L}(\alpha)$ given any input α in Σ^n, using $T'(n)$ steps of chain of thought with probability higher than $2/3$.